

CPE310 — Introduction to Agent-Based Systems

Group Project Proposal

Design of an Agent-Based Financial Fraud Detection and Prevention System

A Multi-Agent Architecture Combining Machine Learning,
Rule-Based Reasoning, and Human-in-the-Loop Escalation

Obafemi Awolowo University, Ile-Ife
Department of Computer Science & Engineering

Group Members: [Add Names & Matric Numbers]

Date: August 2026

0. The 30-Second Pitch (for your group)

Elevator pitch

We're building a team of specialised AI agents that watches transactions the way a fraud desk does: one agent cleans and reads each transaction, two agents independently judge it (one with a trained ML model, one with hard compliance rules), a decision agent combines both verdicts, an action agent blocks or allows the transaction and alerts the user, and a learning agent improves the system over time using analyst feedback. It costs nothing to build — free dataset, free open-source tools, runs on our laptops — and it directly demonstrates every concept from the course: autonomy, asynchronicity, agency, the Perceive-Reason-Act cycle, and collaborative multi-agent systems.

Why this project is a strong choice for our group:

- Directly reuses the course's own case-study pattern (the Multi-Agent Resume Optimizer) — low risk, well-understood template to adapt.
- Zero cost: free public dataset, free open-source Python libraries, free hosting for the demo.
- Naturally splits into 5–6 independent workstreams — easy to divide fairly among group members.
- Produces a genuinely useful, demo-able artefact (a working dashboard), not just a slide deck.
- Rich discussion material for the report and defence: class imbalance, false positives vs false negatives, explainability, bias, and privacy — all things markers like to probe.

1. Problem Statement & Motivation

Financial fraud — card-not-present fraud, account takeover, and increasingly mobile-money and USSD fraud — costs banks and customers billions annually and is a growing concern in the Nigerian financial ecosystem as digital payments expand. Traditional fraud detection relies on static, hand-coded rule engines: fast and predictable, but brittle — a rule that catches today's fraud pattern is useless against tomorrow's.

A single monolithic machine-learning model is an improvement but still has a single point of failure: it cannot explain itself well, cannot easily combine statistical evidence with regulatory rules, and cannot gracefully hand off uncertain cases to a human. This project proposes an agent-based alternative: a small team of cooperating, specialised software agents — each autonomous, each reactive to events, each contributing a different kind of reasoning — that together detect, explain, and act on suspicious transactions in real time.

2. Why a Multi-Agent Approach?

This maps directly onto concepts from Part 1, 2 and 4 of the course:

- Nwana's characteristics (autonomy, reactivity, proactivity, social ability) apply to each agent individually — every agent perceives its own inputs and acts without being micromanaged.
- The Three Pillars (Autonomy, Asynchronicity, Agency) describe exactly how the agents must operate: independently, event-driven, and with delegated intent to protect the customer's account.
- The Perceive → Reason → Act cycle is the mental model for every agent in the pipeline (see Section 4).
- Part 4's argument for Collaborative Agents applies precisely: a single agent excels at one task, but fraud detection needs statistical pattern recognition, rule compliance, and judgement — different kinds of expertise working in parallel, exactly like the course's NLP + Data Mining + Reasoning agent example.

Key argument to defend in the viva

“Why not just train one big model?” — Because a single model conflates statistical anomaly with regulatory violation, gives no natural way to weigh conflicting signals, and offers no clean audit trail. Splitting responsibilities across agents gives depth (each agent is good at one thing), resilience (one agent failing doesn't collapse the system), and explainability (each agent's verdict is inspectable).

3. Proposed System Architecture

The system is composed of six agents coordinated by a lightweight orchestrator (message bus), following the Actor-Model pattern discussed in Part 1 of the course.

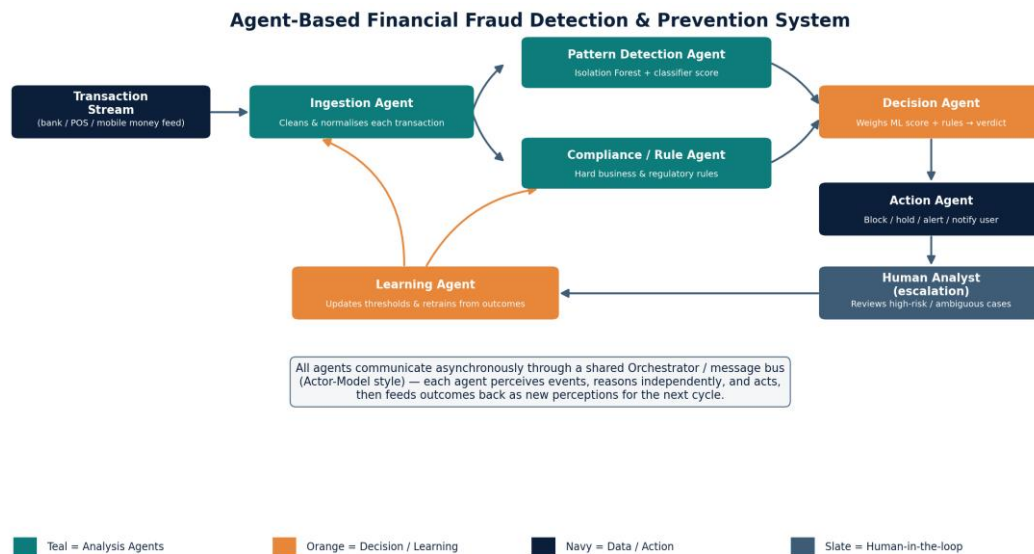


Figure 1: Multi-agent fraud detection architecture

Agent	Responsibility	Course Concept Demonstrated
Ingestion Agent	Receives raw transaction events, validates and normalises fields (amount, location, device, timestamp) into a structured belief.	Perception Module
Pattern Detection Agent	Scores the transaction using an unsupervised anomaly detector and a supervised fraud classifier.	Cognitive Module — statistical reasoning
Compliance / Rule Agent	Applies deterministic business & regulatory rules (velocity limits, blacklists, geo-mismatch, new-device thresholds).	Traditional symbolic agent reasoning
Decision Agent	Combines the ML score and rule flags, resolves conflicts, and issues a verdict: allow / hold / block / escalate.	Decision-Making Submodule (arbitration)
Action Agent	Executes the verdict: blocks or releases the transaction, sends an alert/SMS/email, writes the case log.	Action Module
Learning Agent	Collects analyst feedback on escalated cases and periodically retrains the model / adjusts thresholds.	Learning Module — short & long-term memory

An optional seventh role — the Human Analyst — represents the human-in-the-loop escalation path for cases the Decision Agent cannot resolve with confidence, matching the course's emphasis on augmentation rather than full replacement (Part 2, Human-Agent Collaboration).

3.1 Perceive → Reason → Act, per agent

Stage	What happens
Perceive	Ingestion Agent converts a raw event into structured beliefs (amount, merchant, device fingerprint, geo-location, time since last transaction).
Reason	Pattern Detection & Rule Agents evaluate those beliefs against learned patterns and fixed rules; Decision Agent weighs both into a single verdict.
Act	Action Agent executes the verdict and the outcome (confirmed fraud / false alarm / legitimate) becomes a new perception for the Learning Agent, closing the feedback loop.

4. Technology Stack (All Free / Open-Source)

Every component below can run on a personal laptop at zero cost, which was our key constraint.

Layer	Tool	Why / Cost
Language	Python 3	Free, standard for ML coursework.
ML models	scikit-learn (Isolation Forest, Random Forest / XGBoost)	Free, no GPU required, runs on any laptop.
Agent orchestration	CrewAI or LangGraph (or hand-rolled Python classes + a queue)	Open-source; makes the “multi-agent” framing concrete for grading.
Natural-language explanations	Groq free-tier API or a local small model via Ollama	Zero cost; generates human-readable reasons for each flag.
Storage	SQLite	Free, file-based, zero setup — logs every transaction & verdict.
Dashboard / demo	Streamlit	Free; deploys free on Streamlit Community Cloud for a shareable link.
Version control	GitHub (free tier)	For team collaboration and submission.

5. Dataset

- Primary: Kaggle “Credit Card Fraud Detection” dataset — 284,807 anonymised European transactions with 492 confirmed frauds. Free download, no application required, and it is the standard academic benchmark for this exact problem.
- Optional extension: IEEE-CIS Fraud Detection dataset (Kaggle) if the group wants richer features (device, card, email-domain signals) for the Rule Agent to work with.

Both datasets are heavily imbalanced (fraud is <1% of transactions) — this is a feature, not a bug, for our report: it lets us discuss precision/recall trade-offs and why plain accuracy is a misleading metric, which is good technical depth for the write-up and viva.

6. Suggested Division of Labour

The architecture splits naturally into independent workstreams — a practical reason this project works well for a group, since each member can own an agent end-to-end (build it, test it, explain it in the report) without blocking teammates until the final integration stage.

Workstream	Owns	Deliverable
Data & Ingestion	Dataset preparation, cleaning, Ingestion Agent	Cleaned dataset + ingestion module
ML / Pattern Detection	Isolation Forest + classifier, Pattern Detection Agent	Trained model + evaluation metrics
Rules & Compliance	Business rule set, Compliance Agent	Rule engine + documented rule list
Decision & Orchestration	Decision Agent, agent-to-agent messaging / orchestrator	Working end-to-end pipeline
Action & Learning	Action Agent (alerts/logging), Learning Agent (feedback loop)	Alerting module + retraining script
Dashboard & Report	Streamlit UI, diagrams, final report & slides	Demo-able app + written report

For a group of 4, workstreams can be merged (e.g. one person covers Rules & Compliance together with Decision & Orchestration). For a group of 5–6, each row can be one person.

7. Suggested Timeline

Week	Milestone
1	Finalise proposal, download & explore dataset, assign roles
2	Build Ingestion Agent + data cleaning pipeline; agree on message format between agents
3	Train & evaluate Pattern Detection Agent (Isolation Forest + classifier)
4	Build Compliance/Rule Agent and Decision Agent; wire up orchestration
5	Build Action Agent (alerts/logging) and Learning Agent (feedback loop)
6	Build Streamlit dashboard; end-to-end integration testing
7	Evaluation, write-up, diagrams, rehearse pitch/demo
8	Buffer week + submission

8. Evaluation Plan

Because fraud is rare (<1% of transactions), accuracy alone is misleading — a model that always predicts “not fraud” would be over 99% accurate while catching zero fraud. We will report:

- Precision & Recall on the fraud class specifically
- F1-score and the Precision-Recall curve (more informative than ROC for imbalanced data)
- Confusion matrix, to discuss the real cost trade-off: a false negative (missed fraud) is expensive for the bank; a false positive (blocking a legitimate transaction) is expensive for customer trust

- A short qualitative test: manually inject a few realistic fraud patterns and confirm the agent pipeline flags them with a sensible explanation

9. Anticipated Challenges & Mitigations

Challenge	Mitigation
Severe class imbalance in training data	Use appropriate techniques (class weighting, SMOTE, or anomaly-detection framing) and evaluate with precision/recall, not accuracy.
Explainability of ML verdicts	Decision Agent always attaches the contributing rule flags and top features driving the ML score; optional LLM call turns this into a plain-English reason.
Data privacy & ethics	Use only the anonymised public dataset for development; discuss in the report how a real deployment would need encryption, access control, and audit logging (tie to course's Bias & Transparency / Data Privacy slide).
Scope creep (project too large for the timeline)	Ship a working core pipeline first (Ingestion → Pattern Detection → Decision → Action) before adding the Learning Agent's automatic retraining.

10. Budget

Total cost: ₦0

Dataset: free (Kaggle). Libraries: free & open-source (scikit-learn, CrewAI/LangGraph, Streamlit, SQLite). LLM explanations: free tier (Groq) or fully local (Ollama). Hosting for the demo: free (Streamlit Community Cloud) or run locally on presentation day. Development hardware: personal laptops — no GPU or paid cloud compute required for this dataset size.

11. References

- Nwana, H.S. (1996). “Software Agents: An Overview.” Knowledge Engineering Review 11(3).
- Hewitt, C. et al. (1973). “A Universal Modular ACTOR Formalism for AI.” IJCAI.
- Wooldridge & Jennings (1995). “Intelligent Agents: Theory and Practice.” Knowledge Engineering Review 10(2).
- AWS Prescriptive Guidance (2025). “Foundations of Agentic AI on AWS.”
- Dal Pozzolo et al. Kaggle Credit Card Fraud Detection Dataset — Université Libre de Bruxelles.
- CPE310 Course Slides — Introduction to Agent-Based Systems, OAU Dept. of Computer Science & Engineering.